

Week 1: Context Free Grammars

We start with some revision of sets. If you are unfamiliar with any of the notation, please ask one of the TAs.

- * 1. Write $\mathbb{N}$ for the set of natural numbers $0, 1, 2, \dots$, and write Σ for the alphabet $\{0, 1\}$. List the elements of the following sets in any order.

- (a) $\{1, 2, 3\} \cup \{2, 4, 6\}$
- (b) $\{1, 2, 3\} \cap \{2, 4, 6\}$
- (c) $\{1, 2, 3\} \times \{2, 4, 6\}$
- (d) $\{1, 2, 3\} \setminus \{2, 4, 6\}$
- (e) $\{2m \mid m \in \mathbb{N}, 0 \leq m \leq 5\}$
- (f) $\{uu \mid u \in \Sigma^*, |u| = 2\}$
- (g) $\{u0v \mid u \in \Sigma^*, v \in \Sigma^*, |uv| = 2\}$
- (h) $\{uvw \mid u \in \Sigma, v \in \Sigma, w \in \Sigma, w \text{ is the xor of } u \text{ and } v\}$

Recall the grammar from the notes for Boolean expressions.

$$\begin{aligned} B &::= F \mid F \parallel B \\ F &::= L \mid L \ \&\& \ F \\ L &::= \text{true} \mid \text{false} \mid (B) \end{aligned}$$

- * 2. Give derivations for the following strings which are in the language of Boolean expressions.

- (a) `true && false`
- (b) `true || false && true`
- (c) `true && (false || true)`

- * 3. Give three distinct derivations for the string `true || false`.

- * 4. Figure 1 gives the production rules for the Microbrew language, which we shall discuss further in

$$\begin{aligned}
Cmd &::= \$ \\
&| Exp \$ \\
&| \text{def ident } (ExpList) = Exp \$ \\
Exp &::= \text{var} \\
&| \text{ident } (ExpList) \\
ExpList &::= \epsilon \\
&| Exp ExpList' \\
ExpList' &::= \epsilon \\
&| , Exp ExpList'
\end{aligned}$$

Figure 1: The Microbrew language grammar.

Week 3. The grammar has start symbol *Cmd*, and is built over eight terminal symbols:

var ident () , def = \$

Intuitively, the terminal symbol *ident* represents any valid identifier, and *var* represents any valid variable name. The terminal symbol *\$* is used as an explicit end-of-input marker. Which of the following strings are derivable? You need not give a derivation.

- (a) *ident*(*var*) *\$*
- (b) *ident*(*ident*()) *\$*
- (c) *ident*() *\$*
- (d) *ident*(*ident*, *ident*) *\$*
- (e) *ident*(*var*) = *ident*(*var*) *\$*
- (f) *def var* = *ident*() *\$*
- (g) *def ident*(*var*) = *var* *\$*
- (h) *def ident*() = *var* *\$*
- (i) (*ident*()) *\$*
- (j) (*def ident*() = *var*) *\$*

- ** 5. Consider the following two grammars over the terminal symbols { a, b }, each of which has start symbol *S*.

$S ::= \epsilon \mid aSb$ $S ::= \epsilon \mid a \mid b \mid aSa \mid bSb$

In all of the following questions, strings refer to strings over the alphabet $\{a, b\}$.

- (a) Give three strings that are derivable in the left grammar, but not in the right grammar.
- (b) Give three strings that are derivable in the right grammar, but not in the left grammar.
- (c) Give a new context free grammar which derives exactly those strings that are either derivable in the left grammar, or derivable in the right grammar. For example, all six of your strings from parts (a) and (b) should be derivable in the new grammar.
- (d) Give a new context free grammar which derives exactly those strings that are derivable in both the left and right grammars.
- (e) Give a new context free grammar which derives the following language of strings, in which u^R is the reversal of the string u :

$$\{a^n u b^n \mid u \in \{a, b\}^* \wedge n \geq 0 \wedge u = u^R\}$$

(That is, each string in the language consists of some possibly empty prefix of a , followed by some string u which is its own reverse, followed by a suffix of b which is exactly the same length as the prefix of a .)

- *** 6. An ϵ -production is a rule of shape $X ::= \epsilon$ (for some nonterminal X). A unit production is a rule of shape $X ::= Y$ (for some non-terminals X and Y). Consider the following grammar G with start symbol S :

$$\begin{aligned} S &::= aSbb \mid T \\ T &::= bTaa \mid S \mid \epsilon \end{aligned}$$

This grammar has two unit productions $S ::= T$ and $T ::= S$, and it has an epsilon production $T ::= \epsilon$.

Give a grammar with *no* unit productions and *no* ϵ -productions that derives exactly the non-empty strings that are already derivable in G (i.e. every *non-empty* string derivable in G should be derivable in the new grammar, and every string derivable in the new grammar should be a non-empty string already derivable in G).